

NL2TL-Tree: Natural Language to Nested Temporal-Logic Trees for Robustness-Guided Flow-Matching Planning

First Author
first.author@example.edu
Your Institution
City, Country

Second Author
second.author@example.edu
Your Institution
City, Country

Third Author
third.author@example.edu
Your Institution
City, Country

Fourth Author
fourth.author@example.edu
Your Institution
City, Country

Fifth Author
fifth.author@example.edu
Your Institution
City, Country

Abstract

Letting non-expert users command robots in plain language is a long-standing goal of ubiquitous computing, but free-form instructions are ambiguous and give no guarantee that the resulting behavior is correct. We present **NL2TL-Tree**, an *early-stage prototype* exploring whether a *nested Signal Temporal Logic (STL) tree* can serve as an interpretable, checkable interface between natural-language robot instructions and motion planning. The pipeline is *training-free*—no model is trained or fine-tuned: a frozen LLM is few-shot-prompted to parse a phone-typed task into the tree (human-readable and error-localizable to a single sub-node), and a *frozen* graph-encoded flow-matching planner (TeLoGraF), steered by a differentiable STL-robustness term, turns the tree into motion. Test-time mechanisms (operator normalization, nested-tree decomposition, a feasibility self-check that defers to a classical backend) extend the frozen planner, and an online robustness monitor can trigger a stop/re-plan; a *planning-latency-aware* deadline further debits the planner’s own wall-clock from timed goals. We report *preliminary evidence*—a small parsing pilot and a TurtleBot3 Gazebo *simulation*—and outline an evaluation plan for parsing accuracy, failure localization, and user-facing interpretability. Code, prompts, and configs are released.¹

CCS Concepts

• **Computing methodologies** → **Robotic planning**; *Natural language generation*; • **Human-centered computing** → Human computer interaction (HCI).

¹<https://github.com/gjc823878198/NL2TL-Tree-Natural-Language-to-Nested-Temporal-Logic-Trees-for-Robot-Flow-Matching-Planning>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UbiComp/ISWC '26, Shanghai, China

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/10
<https://doi.org/XXXXXXX.XXXXXXX>

Keywords

natural language interfaces, signal temporal logic, flow matching, diffusion models, training-free LLM parsing, task planning, verifiable AI, human-robot interaction

ACM Reference Format:

First Author, Second Author, Third Author, Fourth Author, and Fifth Author. 2026. NL2TL-Tree: Natural Language to Nested Temporal-Logic Trees for Robustness-Guided Flow-Matching Planning. In *Proceedings of Companion of the 2026 ACM International Joint Conference on Pervasive and Ubiquitous Computing and the 2026 ACM International Symposium on Wearable Computers (UbiComp/ISWC '26)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Pervasive and wearable systems increasingly let people issue tasks in everyday language—“check the kitchen every ten minutes, but never enter the bedroom.” Such instructions are inherently *temporal* and *safety-constrained*, yet most natural-language interfaces translate them with end-to-end models whose output cannot be inspected, checked, or trusted before execution. For ubiquitous computing, where the system acts in a user’s physical space, this lack of interpretability and verifiability is a core barrier to adoption.

Signal Temporal Logic (STL) is an attractive target: it expresses temporal and spatial constraints precisely and admits a quantitative *robustness* of satisfaction. Two threads have grown around it—translating language into formulas [2, 3], and generative planning for STL tasks [7, 9]—but usually in isolation: language work stops at a formula, planning work assumes one is given.

We connect them with one modular pipeline (Fig. 1) built around a single idea—*keep an explicit, verifiable logical structure between the human and the planner*. The entry point is a phone: a user types (or dictates) a task and receives a checked *spatiotemporal command* (where to be, what to avoid, when) that a mobile robot executes. The STL spec enters a frozen flow-matching planner natively through its graph encoder, and a differentiable robustness term steers sampling; nothing is trained, so the train/use gap is closed at test time (§4). In this poster we present the NL2TL-Tree *prototype*—NL → nested STL tree → robustness monitor → frozen flow-matching planner—built around the nested tree as an *interpretable, checkable* intermediate representation, and report a *preliminary feasibility study* (a frozen-LLM parsing pilot and a Gazebo simulation, §4). The

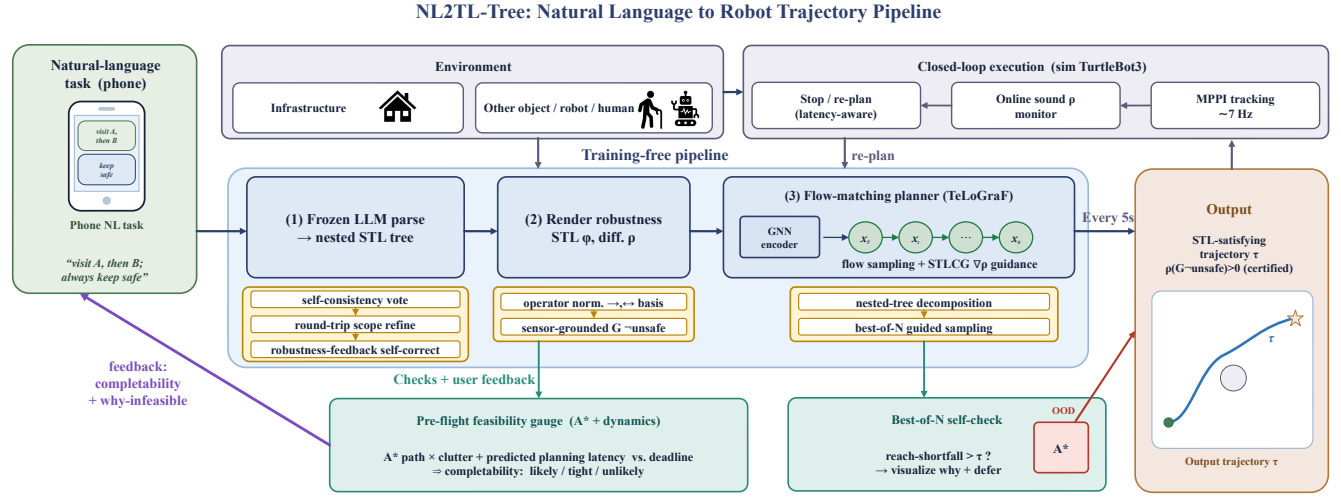


Figure 1: The NL2TL-Tree pipeline—everything frozen / training-free. A phone natural-language task is parsed by a frozen LLM into a nested STL tree, rendered to a differentiable robustness ρ , and planned by a frozen flow-matching planner (TeLoGraF) under STLCG robustness-gradient guidance; the certified trajectory is executed closed-loop on a simulated TurtleBot3 with an online sound- ρ monitor and a planning-latency-aware stop/re-plan. Each stage is annotated with the test-time mechanisms it uses (§4). A *checks-and-feedback* layer—a pre-flight completability gauge (an A^* path + dynamics estimate vs. the deadline) and a best-of- N self-check that defers out-of-distribution specs to an A^* fallback—returns the completability and a brief *why-infeasible* explanation to the user.

structural-accuracy levers (self-consistency, round-trip and robustness-feedback self-correction) and a user study are *ongoing*, not yet evaluated.

2 Approach

2.1 Training-free natural language to a nested STL tree

Rather than emitting a flat formula string, we prompt a *frozen* LLM M to produce a *nested logical tree* serialized as JSON (Fig. 2)—atomic predicates at the leaves, Boolean/temporal operators ($\square$, $\diamond$, $\mathcal{U}$, timed variants) at internal nodes. Unlike a flat formula, this representation is *human-readable*, *grammar-checkable* before execution, and *error-localizable* to a single sub-tree—and a round-trip back to an STL string aids verification (Fig. 2, bottom).

By *training-free* we mean *parameter-update-free* (not data-free): no model’s weights change—the LLM is steered only by K_{shot} in-context exemplars (few-shot, not zero-shot) and the planner stays frozen. The LLM is invoked with a strict-JSON output mode and a prompt assembled from K_{shot} input–output exemplars; we treat K_{shot} as a user-tunable budget ($K_{\text{shot}}=8$ here; larger values trade tokens for coverage of operator diversity). Schema validation (VALIDATESCHEMA) rejects malformed JSON or unknown operators before downstream use.

Robustness-feedback self-correction. Rather than accept the first parse, we *verify* it and re-prompt the same frozen LLM when a check fails—a neuro-symbolic loop with no parameter updates. Three verifiers fire cheapest-first: (1) operator arity/schema; (2) grounding consistency (every atom names a known place, no place

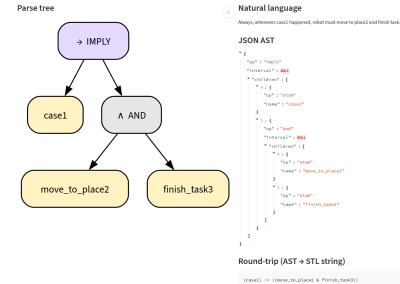


Figure 2: The interpretable intermediate representation (here in our visual inspection tool). A natural-language instruction is parsed into a nested STL tree (left), serialized as the JSON AST that downstream stages consume (right), and round-tripped to an STL string for verification (bottom). The explicit tree exposes operator scope at a glance—the structure the parser must get right and that self-correction repairs.

is both reach and avoid); and (3) *quantitative STL robustness*—we plan a trajectory and, if its robustness ρ cannot be made positive, the spec is unsatisfiable/contradictory and we feed ρ and the offending sub-tree back as a targeted correction. Each failed round appends the rejected JSON plus the diagnostic to the prompt and re-queries (bounded rounds). This directly attacks our dominant error mode (right operators, wrong nesting; see §4); it is implemented, but its quantitative gain is pending the larger study.

Structural-accuracy levers (training-free). Since the residual error is almost purely *structural* (§4), we add two mechanisms that target

the nesting. (i) **Structural self-consistency**: sample K parses at temperature >0 , canonicalize each tree, and majority-vote the structure [13]—operators barely vary across samples while scope does, so the majority recovers nesting that greedy decoding loses to a single near-miss. (ii) **Round-trip scope refinement**: the tree is paraphrased back to NL with its scope made explicit and regenerated only when it disagrees with the instruction [10], turning an invisible inversion into a checkable one.

Role of the public NL-STL dataset. We use a public NL-to-STL dataset three frozen ways: a *few-shot pool* (K_{shot} exemplars); a source of *gold trees* (formulas converted once off-line by a Lark grammar into evaluation-only references); and a *reservoir* of mis-parses written back as future exemplars—none trains a model.

2.2 Rendering to differentiable robustness

The tree compiles to a canonical STL formula φ and a *differentiable robustness* $\rho(\cdot; \varphi)$ under the standard quantitative semantics ($\rho(\neg) = -\rho$, $\rho(\wedge) = \min$, $\rho(\diamond) = \max_t$, dually for $\square, \mathcal{U}$; $\rho > 0 \Leftrightarrow x \models \varphi$; Fig. 3). Following masking-based formulations [8], this smooth ρ is an *optimizable surrogate* that steers sampling; a *separate sound monitor* (exact, non-smoothed) does the validation, with a *duration-aware* keep-safe robustness [6] (worst-case clearance on a satisfied path; penalizing unsafe duration under violation). Optimizer and judge are different functions. We anchor the index axis to wall-clock seconds (arclength at a nominal speed), so $F_{[0,D]} \text{REACH}$ is a real *deadline*; the monitor validates keep-safe and reach-by-deadline on the executed trace. Both margins are in *metres*: ρ_{safe} is worst-case clearance, ρ_{time} the robustness of $F_{[0,D]} \text{REACH}$ within the deadline window (>0 iff the goal is entered before D). Run *online*, a non-positive ρ stops and re-plans (a runtime monitor in simulation), so a violation is repaired, not merely logged.

Planning-latency-aware deadline. The flow-matching planner is not instantaneous (cold-start, then seconds per re-plan), and that wall-clock elapses against $F_{[0,D]}$ —the naive ρ_{time} *freezes* time while the planner thinks. We predict its latency online (cold-start-seeded EMA) and *debit* it: $\rho_{\text{time}}^{\text{lat}} = (D - \hat{t}_{\text{plan}}) - t_{\text{exec}} \leq \rho_{\text{time}}$, handing the tightened deadline to the next re-plan; a plan that “arrives in time” ignoring its own planning cost is then flagged late.

Sensor-grounded keep-safe. Obstacle avoidance is *one* clause $\square \neg \text{UNSAFE}$, with UNSAFE grounded by the union of LiDAR-sensed disks $\{(c_k, r_k)\}$, so $\rho(\square \neg \text{UNSAFE}) = \min_t \min_k (\|p_t - c_k\| - r_k)$ is the worst clearance. A union predicate (vs. N avoid atoms) keeps the GNN encoder in-distribution while the *same* sensed set grounds the keep-safe semantics and drives guidance—so what the robot senses becomes its keep-safe constraint, monitor-validated online ($\rho > 0$) under the sensed obstacle model.

2.3 Tree to trajectory: a frozen flow-matching planner

We pair ρ with a *frozen* generative planner—concretely TeLoGraF [12], which ingests the STL graph through a GNN encoder (the spec enters natively as conditioning) and samples trajectories by flow matching; a differentiable ρ term steers the sampler toward high-satisfaction regions (Fig. 3). The recipe also fits score-based backbones [1, 4, 7, 11]; weights are never updated. **Three test-time**

mechanisms make the frozen planner usable on real specs (Fig. 3), all leaving p_θ untouched. (i) **Operator normalization**: the graph vocabulary lacks $\rightarrow, \leftrightarrow$, so we rewrite them with the standard identities $A \rightarrow B \equiv \neg A \vee B$ and $A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$, verified to preserve robustness *exactly* (zero deviation over thousands of traces and a Boolean truth-table check). (ii) **Nested-tree decomposition**: a nested sequential spec is out-of-distribution, yet each leg is an in-distribution timed reach, so we split the tree, flow-match each leg, and stitch (cf. [5, 9]). (iii) **Feasibility self-check**: from the spread of the best-of- N prior’s reach-shortfall—an *empirical* OOD signal that separates our flow-feasible specs from the out-of-distribution nested one—we defer to a classical planner only when even the best candidate is beyond guidance’s reach (best-of- N per-atom reach-shortfall $> \tau_{\text{OOD}} = 2.5$ m), making the generative-vs-classical choice *measured*, not fixed. The same A^* /dynamics estimate also gives a **pre-flight completability** check—an A^* path through the goals, inflated for clutter and *charged the predicted planning latency*, timed against the deadline—flagging an unreachable deadline up front rather than mid-run.

3 Design Rationale

The pipeline is *designed* for four structural properties, though whether they yield measurable *user* benefits (faster error-spotting, calibrated trust) is a hypothesis we have *not yet user-validated* (§5). It is *interpretable* (the human-planner interface is an explicit nested tree, so a wrong instruction localizes to one sub-node), *checkable* (the sound monitor of §2.2 checks the executed run at the monitor level, $\rho \leq 0$ triggering a stop/re-plan in simulation, with intent aligned by the round-trip check), *training-free* (LLM and planner stay frozen), and *modular* (the typed $\text{NL} \rightarrow \text{tree} \rightarrow \rho \rightarrow \text{trajectory}$ interfaces let any stage be swapped). Unlike NL2TL [3], which fine-tunes a model and stops at a *flat* formula, NL2TL-Tree is training-free, emits a *checkable nested tree*, and carries the spec through to execution; and unlike the planner we reuse—TeLoGraF [12], which assumes the STL is given and is tied to a fixed operator set—we keep it *frozen* and extend it purely at test time.

The user-facing loop. The tree is the interaction surface (Fig. 2): a user types a task on a phone, sees the parsed nested tree rather than an opaque trajectory, localizes a mis-scoped constraint to one sub-node, edits or re-prompts, and reads a keep-safe / reach-by-deadline *monitor* value ($\rho > 0$; a monitor-level check, not a system-wide guarantee) *before* the robot moves—an *interactive, checkable command interface* rather than a black box. Whether it speeds error-spotting for non-experts is left to a formative study (§5).

4 Case Study and Evaluation

We report a *preliminary* case study (not a full benchmark): a parsing study with a frozen LLM (Stage 1) and an end-to-end planning evaluation in TurtleBot3 *simulation* (Gazebo Classic) (Stage 2).

4.1 Stage 1: NL $\rightarrow$ STL tree (frozen LLM)

Data + protocol. We use the public NL2TL “lifted” dataset (predicates abstracted to $\text{prop}_1, \dots$), covering all eight STL operators and their unbounded variants [3]; each row pairs a sentence with a gold STL formula, which a Lark grammar converts once into a gold tree. We call the frozen LLM in strict-JSON mode (zero training;

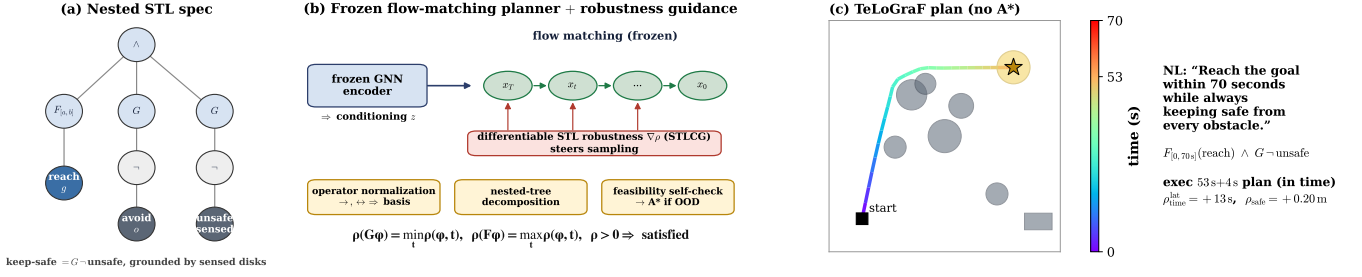


Figure 3: From spec to trajectory (§2.2–2.3). (a) the nested STL tree, with reach (blue) and avoid / keep-safe (grey) atoms; (b) a frozen GNN encoder conditions a flow-matching sampler ($x_T \rightarrow x_0$) that the differentiable STL-robustness gradient $\nabla \rho$ steers, plus the three test-time mechanisms (operator normalization, nested-tree decomposition, feasibility self-check $\rightarrow A^*$); (c) one deployed timed task solved by pure frozen TeLoGraF (no A^*)—the trajectory, coloured by wall-clock seconds, enters the goal at 53 s of a 70 s deadline (latency-aware $\rho_{\text{time}}^{\text{lat}} = +13\text{ s}$, keep-safe $\rho_{\text{safe}} = +0.20\text{ m}$). Per-task results: Table 1.

Table 2); it never sees the formula, and its predicted tree is scored directly against the gold tree. We score the first 30 distinct sentences (no cherry-picking; every parse, including errors, is counted).

Metrics. Beyond strict equality: *op-F1* (operator multiset—right pieces?), *path-F1* (root-to-leaf path multiset—right nesting?), and normalized tree-edit distance ($1 - \text{TED} / \max(|T_p|, |T_g|)$).

Result (preliminary pilot, $n=30$, no fine-tuning). As an initial feasibility check, the frozen LLM usually picks the right operators (**op-F1**=0.95) but *mis-nests scope* (**path-F1**=0.64); exact-match is 56.7% (95% Wilson CI [39, 73]—wide, as expected for a pilot) and the average tree is only 1.03 edits from gold ($\text{TED}_{\text{norm}}=0.84$). The error is thus almost entirely *structural*, which motivates the structural self-consistency voting [13] and round-trip scope refinement [10] levers. A small ablation ($n=30$): self-consistency ($K=5$) helps modestly (exact 0.57 $\rightarrow$ 0.60), while round-trip refinement did *not* (it lowered exact-match). These pilot figures are indicative, pending a larger evaluation.

Flat-STL baseline. A flat-STL baseline (the same frozen LLM emits an STL *string*, parsed by the same grammar; same sentences/gold/metrics) is *no more accurate*: across two further open LLMs the tree matches or beats it on op-F1 (0.94–0.96 vs. 0.90–0.92) and path-F1, trailing only slightly on exact-match (0.60–0.63 vs. 0.67, both grammar-canonicalized). The interpretable structure is essentially free.

What the study stresses. These pilot scores matter less than the *test-time machinery* of §2. Parts of it appear in Fig. 1 but are only sketched here—the *why-infeasible* feedback and the best-of- N self-check visualization—and run live in our demo supplement.

4.2 Stage 2: tree $\rightarrow$ trajectory

We evaluate the frozen-TeLoGraF planner two ways, as an illustrative case study rather than a generalization benchmark.

(i) *Per-task study (cluttered scene; Table 1, Fig. 3(c)).* On six *hand-built* tasks spanning the STL templates, the sound monitor reports $\rho(\Box \neg \text{UNSAFE}) > 0$ and goal entry. An *ablation* (3 re-runs/task): pure flow + STLCG solves the four unnested timed-reach tasks (4/6) but not the nested sequential or OOD response; the full pipeline recovers both (6/6), and disabling the A^* repair drops back to 4/6—the *repair is the load-bearing recovery*, decomposition alone leaving the

Table 1: Per-task Stage-2 outcome on the cluttered scene of Fig. 3 (six hand-built tasks). On all six the sound monitor reports keep-safe $\rho(\Box \neg \text{UNSAFE}) > 0$ and goal entry; ρ_{safe} is the mean keep-safe margin over 3 re-runs (full pipeline). The engine is selected automatically: “flow” = pure TeLoGraF + STLCG (no A^*), “+decomp.” = per-leg flow, “ A^* ” = classical fall-back. The lone out-of-distribution spec (trigger_response) is routed to A^* —by design, not a silent success.

Task	STL template	Planner	ρ_{safe} (m)
reach_within_T	$F_{[a,b]} g \wedge \Box \neg u$	flow	+0.20
reach_avoid	$F g \wedge \Box \neg o$	flow	+0.20
reach_goal_north	$F g \wedge \Box \neg o$	flow	+0.20
seq_reach_ABC	$F(A \wedge F(B \wedge F C))$	decomp + A^*	+0.27
cond_reach_either	$\neg F N \rightarrow F S (\equiv F N \vee F S)$	flow	+0.20
trigger_response	$\Box(p \rightarrow F_{[0,5]} q)$	A^* (OOD)	+0.21

count unchanged. Operator normalization separately lets pure flow discharge an implication spec (cond_reach_either). (ii) *Simulated closed loop (TurtleBot3, Gazebo Classic; full setup in Table 2).* On live LiDAR it reaches the goal in 35/53 s ($\rho_{\text{time}} = +0.1$, keep-safe +1.2 m); a *multi-goal* variant visits three regions in order through a cylinder field, all in time.

Metrics. Beyond satisfaction and robustness ρ , we report *inference time*: each re-plan completes in 3.6–4.3 s (mean 3.9, $n=6$) on a laptop CPU—the very latency our planning-aware $\rho_{\text{time}}^{\text{lat}}$ debits (§2.2). Failures are detected, not hidden: the post-hoc check routes a failing plan to A^* repair.

Planned baselines. A study (*not yet run*) will compare against classical LLM $\rightarrow$ STL $\rightarrow$ solver [2], generative [1, 7], and STL/LTL-aware [4, 5, 9, 11, 12] planners.

5 Discussion and Future Work

The structured, training-free middle layer is a promising abstraction for NL robot control: the nested tree is *checkable* and error-localizable to one sub-node, at no parsing-accuracy cost in our pilot. Whether it helps *people* (faster error-spotting, calibrated trust) is the open question our design targets but has not yet user-validated. **Future work** spans a larger parsing benchmark, a Stage-2 baseline study, a formative user study, real-LiDAR hardware, and a better-matched TL robustness.

A Reproducibility

The released code, prompts, and configs (link in the abstract) reproduce the study; Table 2 lists the settings used throughout.

Table 2: Reproducibility settings (frozen LLM + frozen planner).

<i>Parsing (frozen LLM)</i>	
LLM	Llama-3.3-70b-versatile (Groq), strict-JSON
Decoding	greedy (temperature 0), $K_{\text{shot}}=8$
Self-consistency	K samples at temperature 0.7, majority vote
<i>Planning (frozen TeLoGraF)</i>	
Checkpoint	simple_gnn_F (frozen)
Sampling	best-of- N ($N \in [16, 64]$; 8 closed-loop), 64 steps
Guidance	STLCG softmin/softmax $\nabla \rho$
OOD self-check	defer to A* if shortfall $> \tau_{\text{OOD}}=2.5$ m
<i>Closed loop</i>	
Re-plan	every 5 s (MPC-style)
Tracking	MPP1 ~ 7 Hz, $v_{\text{max}}=0.22$ m/s
Monitor	stops / re-plans on $\rho \leq 0$ (monitor-level)
<i>Scene</i>	
Goals	reach disks $r \approx 0.45$ m
Obstacles	LiDAR-sensed cylinders $r=0.22$ m

Acknowledgments

We thank our advisor for invaluable guidance, and our senior lab-mates for their generous support and help throughout this work.

References

- [1] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua Tenenbaum, Tommi Jaakkola, and Pulkit Agrawal. 2023. Is Conditional Generative Modeling All You Need for Decision-Making?. In *International Conference on Learning Representations (ICLR)*.
- [2] Yongchao Chen, Jacob Arkin, Yang Zhang, Nicholas Roy, and Chuchu Fan. 2024. AutoTAMP: Autoregressive Task and Motion Planning with LLMs as Translators and Checkers. In *IEEE International Conference on Robotics and Automation (ICRA)*.
- [3] Yongchao Chen, Rujul Gandhi, Yang Zhang, and Chuchu Fan. 2023. NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [4] Zeyu Feng, Hao Luan, Pranav Goyal, and Harold Soh. 2024. LTLDoG: Satisfying Temporally-Extended Symbolic Constraints for Safe Diffusion-based Planning. *IEEE Robotics and Automation Letters* 9, 10 (2024), 8571–8578.
- [5] Zeyu Feng, Hao Luan, Kevin Yuchen Ma, and Harold Soh. 2025. Diffusion Meets Options: Hierarchical Generative Skill Composition for Temporally-Extended Tasks. In *IEEE International Conference on Robotics and Automation (ICRA)*. 10854–10860.
- [6] Florian Finkeldei, Michael Wolf, Jan-Niklas Weghorn, Alexander Pretschner, and Matthias Althoff. 2025. Enhanced Traffic Rule Monitoring Using Model-Predictive and Duration-Aware Robustness. In *IEEE International Conference on Intelligent Transportation Systems (ITSC)*. doi:10.1109/ITSC60802.2025.11423597
- [7] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. 2022. Planning with Diffusion for Flexible Behavior Synthesis. *arXiv preprint arXiv:2205.09991* (2022).
- [8] Parv Kapoor, Kazuki Mizuta, Eunsuk Kang, and Karen Leung. 2025. STLCG++: A Masking Approach for Differentiable Signal Temporal Logic Specification. *IEEE Robotics and Automation Letters* 10, 9 (2025), 9240–9247.
- [9] Ruijia Liu, Ancheng Hou, Xiao Yu, and Xiang Yin. 2025. Zero-Shot Trajectory Planning for Signal Temporal Logic Tasks. *arXiv preprint arXiv:2501.13457* (2025).
- [10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2303.17651.
- [11] Yue Meng and Chuchu Fan. 2024. Diverse Controllable Diffusion Policy with Signal Temporal Logic. *IEEE Robotics and Automation Letters* 9, 10 (2024), 8354–8361.
- [12] Yue Meng and Chuchu Fan. 2025. TeLoGraF: Temporal Logic Planning via Graph-encoded Flow Matching. *arXiv preprint arXiv:2505.00562* (2025).
- [13] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2203.11171.